

3.1 Input: st (stack), Max st, Item, top
 output: item in stack if stack not overflow

If Top = Maxst then
 print stack overflow;

Else

Top = Top + 1

st[Top] = Item;

End IF

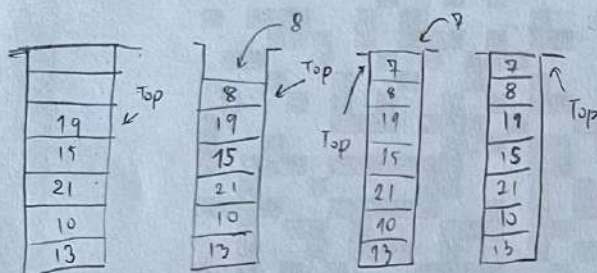
1
 -1 → 2

-1 → 3 ✓

fcn = 3

fcn = O(1)

เมื่อมันเต็มก็ใส่ (worst case).



3.2 public class StackExam1
 int Maxst = 7;
 int st[] = new int[Maxst];
 int Top = -1;
 int Item;

public int Pop() {

if (Top == -1) {

System.out.println("stack is Empty");

}

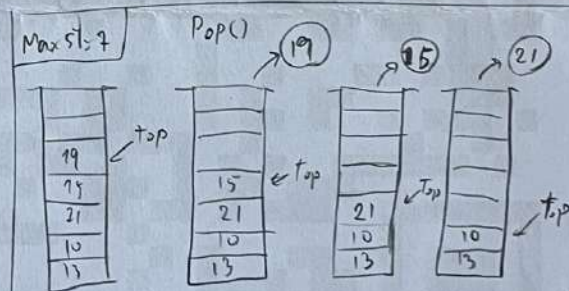
Item = st[Top];

Top--;

return Item;

}

public void push(int t) {



fcn = 4
 fcn = O(1)

7. အဲဒါ Node B ဖြစ်

7.1 B.Link ဖြစ် C

7.2 C.Back ဖြစ် B

7.3 start.Link ဖြစ် B

7.4 ဒီလို ခုတ်ပစ်ပစ်

8. အဲဒါ Node A ဖြစ်

8.1 ဒီလို ခုတ်ပစ်ပစ် A

8.2 A.Link ဖြစ် B

8.3 B.Back ဖြစ် A

8.4 start.Link ဖြစ် A

5.2 Input: List, PTR, start, Item, Loc

Output: Loc (အမှတ်အသား Item)

PTR = start

while PTR != NULL do

if Item <= INFO[PTR] then

PTR = LINK[PTR]

else

if Item <= INFO[PTR] then

Loc = PTR

Return Loc

else

Loc = NULL

Return Loc

endif

endif

Loc = NULL

End While

Return Loc

$$f(n) = 5n + 2$$
$$f(n) = O(n)$$

```

5.1 public class Node {
    Object INFOR;
    Node LINK = null;
    Node Back = null;
}

```

ขั้นตอนการดำเนินการ

5.1

1. สร้าง Node start ขึ้น
2. สร้าง Node Last ขึ้น
3. สร้าง Node F ขึ้น
 - 3.1 ให้ start LINK เป็น F
 - 3.2 start LINK เป็น F
 - 3.2 Last Back เป็น F
4. สร้าง Node E ขึ้น
 - 4.1 ให้ start LINK เป็น E
 - 4.2 E LINK เป็น F
 - 4.3 start LINK เป็น E
 - 4.4 A Back เป็น E
5. สร้าง Node D ขึ้น
 - 5.1 ให้ start LINK เป็น D
 - 5.2 D LINK เป็น E
 - 5.3 E Back เป็น D
 - 5.4 start LINK เป็น D
6. สร้าง Node C ขึ้น
 - 6.1 ให้ start LINK เป็น C
 - 6.2 C LINK เป็น D
 - 6.3 D Back เป็น C
 - 6.4 start LINK เป็น C

โครงสร้างข้อมูล สัมพันธ์กับ อัลกอริทึม

โครงสร้างข้อมูล $\rightarrow$ วิธีการจัดเก็บ ข้อมูลในหน่วยความจำ.

อัลกอริทึม $\rightarrow$ ขั้นตอนหรือกระบวนการแก้ปัญหาต่างๆ เช่น Searching, sorting, travel. ฯลฯ

ซึ่งทั้งสองอย่างนี้ ใช้ในการทำงานร่วมกัน เพราะโครงสร้างข้อมูล ส่งผลต่อประสิทธิภาพในการทำงานและการใช้หน่วยความจำ เช่น การค้นหา (Searching) ถ้าใช้ Array จะค้นหา $O(n)$, ถ้าเป็น tree จะใช้ $O(\log n)$

หรือ การใช้ stack กับ อัลกอริทึม ถ้าเราไม่มี stack จะทำให้การเก็บข้อมูลเพื่อ Reverse นั้นต้องใช้ อัลกอริทึมที่มาก ซับซ้อนมากขึ้น.

1. โครงสร้างข้อมูล ที่ศึกษา Array Queue Stack Linked List

1.1 Array คือ หน่วยความจำ ที่จัดระเบียบไว้กับข้อมูล โดยจะจัดเป็นชุดตัวแปรหนึ่งๆ เช่น

int a[] แสดงให้เป็นกลุ่มข้อมูลที่เป็น integer (จำนวนเต็ม). ซึ่งตัว a จะต้องประกาศ int a[] ไว้แล้ว
และค่าไปใช้กำหนดได้อย่างชัดเจน เช่น for loop Array เป็นต้น แต่ข้อเสียคือ ไม่สามารถปรับค่า
ได้ในภายหลัง เช่น การเก็บคะแนน ของนักเรียน ในห้อง เป็นต้น Array สามารถที่จะจัดข้อมูลได้รวดเร็ว

1.2. Queue คือ เป็นโครงสร้างข้อมูลที่ทำงานแบบ หนึ่งก่อน หนึ่งหลัง (FIFO) โดย FRONT เป็นตัวชี้ ตำแหน่งแรก
ที่พร้อมจะออกได้ และ REAR เป็นตัวชี้ ข้อมูลสุดท้ายที่สามารถนำออกได้.

ซึ่ง Queue มีทั้งแบบ เชิงตรง , วงกลม , สองทาง , ฯลฯ

การใช้ Queue เช่น การต่อแถวซื้อตั๋ว , หรือ การจัดลำดับการทำงาน เป็นต้น

1.3. stack คือ ที่เก็บข้อมูลที่ไว้แบบ LIFO (Last in First out). ซึ่งในการใช้งานต้องนำ
แล้วการจะหยิบใช้ ต้องหยิบจากใบบนสุด หรือ ใบที่วางลงมาล่าสุดก่อน.

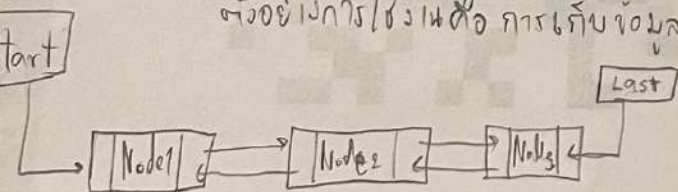
ซึ่งจะมี 2 สกนพจน์ไปไม่ได้ คือ pop (push ใต้) , push (push ใต้) , pop (pop ใต้)
stack สามารถทำได้ 2 อย่างหลักๆ คือ POP (นำข้อมูลล่าสุดออก) , PUSH (เพิ่มข้อมูลเพิ่มไป).

1.4. Linked List มีลักษณะคือ ภาชนะการโย เป็นโครงสร้างข้อมูลที่มี 2 ส่วนย่อยคือ Node (โหนด)

เชื่อมต่อกันเป็นลำดับผ่านตัวชี้ (Link) ไปที่ตำแหน่ง Node ถัดไป และตัวชี้ ไปที่ตำแหน่งก่อนหน้า.

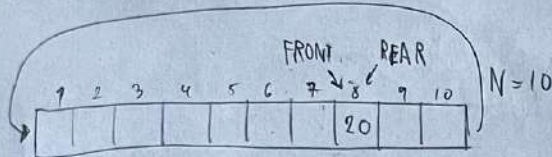
ข้อดีคือ สามารถเพิ่ม/ลด ข้อมูลได้จากทุกจุดได้อย่างรวดเร็ว เป็นที่นิยมในปัจจุบัน

ตัวอย่างการใช้งานคือ การเก็บข้อมูล หนังสือ ที่หนังสือเล่มใหม่จะมาเรียงแทรกตรงกลางได้ เป็นต้น



tree เป็นโครงสร้างการเก็บข้อมูลที่ใช้ในการเก็บแบบเรียงลำดับได้ดี มีการค้นหาที่เร็ว
มีหลักการทำงานข้อมูล 3 แบบ Pre, Post, IN. ใช้ภาพเป็นต้นไม้

4.1



4.2 1. array queue ka kisi bhi array

2. index REAR par 8 index

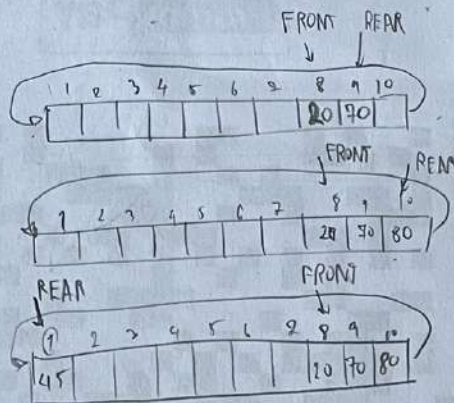
3. Queue[REAR] = 70

4. index REAR par 9 index

5. Queue[REAR] = 80

6. index REAR par 10 index (array)

7. Queue[REAR] = 45



Input: Queue (no), FRONT, REAR, N, Item

Output: Item add in Queue if Queue is not Full

IF (FRONT == 1 and REAR == N) or (FRONT == REAR + 1) then -1

Print ("Queue is Full");

Else IF

IF (FRONT == NULL) then

FRONT = 1

REAR = 1

Else

IF (REAR == N) then

REAR = 1

Else IF

REAR = REAR + 1

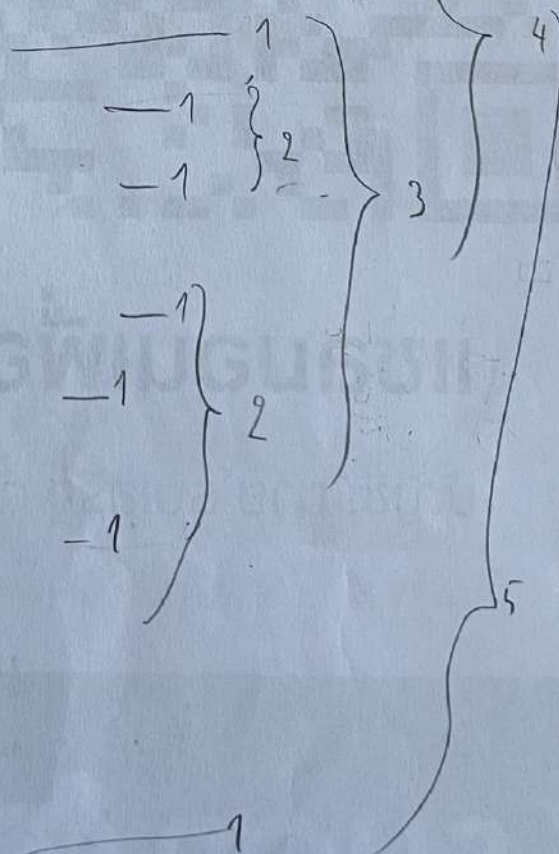
End IF

End IF

End IF

Queue[REAR] = Item

Return



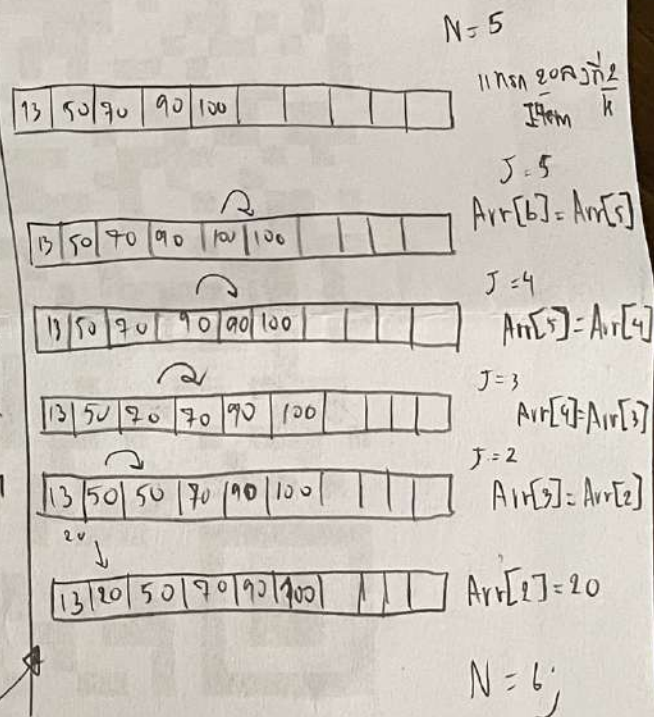
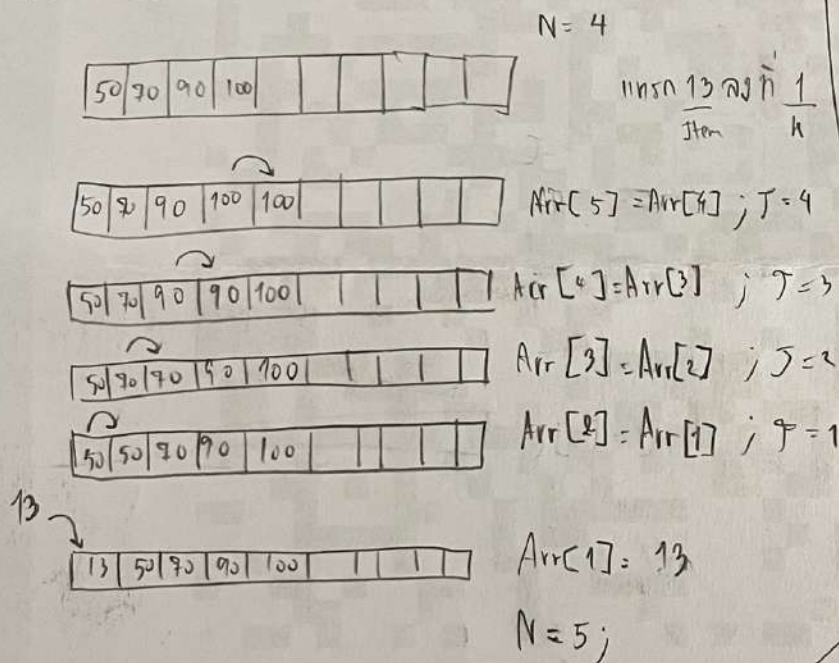
2. Input : Arr , N, K, Item

output : Item or insert in Arr at k

1. $J = N;$ 1
 While $J \leq K$ Do :
 $Arr[J+1] = Arr[J];$ -1
 $J = J-1;$ -2
 End While
 $Arr[K] = Item;$ 2
 $N = N+1;$ 3

$T(n) = 2n+3$
 $= O(n)$

အကယ်၍



အကယ်၍

